

AGENTS

INHERITED FROM the Helix Constitution

Base agent rules live in the Helix Constitution. **READ THOSE FIRST.** The constitution's `AGENTS.md` and the universal `Constitution.md` it references are authoritative for any topic not covered here. Locate the constitution from any nested depth via its `find_constitution.sh` helper — do NOT hardcode a path (this module stays fully decoupled and project-agnostic per §11.4.28). Module-specific rules below extend the universal clauses; they never weaken them.

Critical universal rules every CLI agent (Claude Code, Cursor, Aider, Codex, Gemini CLI, Qwen Code) MUST honour while working in this module:

- **No bluffing.** Every PASS carries positive captured evidence. §11.4.
- **Mutation-paired gates.** Every new gate has a paired mutation proving it catches regressions. §1.1.
- **No guessing language** (likely, probably, maybe, seems). §11.4.6.
- **Credentials never tracked.** `.env` patterns git-ignored; runtime-load only. §11.4.10.
- **Never force-push.** Force-push is forbidden; integrate by merging onto the latest main. §9 / §11.4.113.
- **60% RAM cap.** Heavy work wrapped in a bounded execution scope. §12.6.

Canonical reference: <https://github.com/HelixDevelopment/HelixConstitution>

AGENTS.md — HelixQA Authoritative Agent Guide

Scope: All AI agents, human contributors, and automated processes working on the `digital.vasic.helixqa` module.

Module Overview

HelixQA is an **anti-bluff QA orchestration framework** written in Go (`digital.vasic.helixqa`, go 1.26). It executes YAML test banks across Android, Android TV, Web, and Desktop targets with real-time crash/ANR detection, step-by-step evidence collection, and automated Markdown ticket

generation. It also provides an LLM-driven **Autonomous QA Session** that navigates running applications, verifies documented features, discovers bugs via computer vision, and produces QA reports with video evidence.

Because the entire purpose of HelixQA is to prove that *other* software genuinely works for end users, the framework holds itself to the same bar: every PASS it emits — and every PASS in its own test suite — **MUST** carry positive runtime evidence. CI green is necessary, never sufficient.

Key Capabilities

- **Cross-platform testing** — Android, Android TV, Web, Desktop.
- **Android TV Channels testing** — generic framework for Home Screen Channels (default channel, category channels, Watch Next, deep links).
- **Real-time crash detection** — ADB-based Android crash/ANR detection, browser and JVM process monitoring.
- **Step-by-step validation** — evidence collected at every step to prevent false positives.
- **YAML test banks** — platform-targeted test cases with priority and documentation references (banks/).
- **Evidence collection** — screenshots, logcat, video recording, stack traces, centralized under the output directory.
- **Markdown ticket generation** — auto-generated issue tickets with full evidence for AI fix pipelines.
- **Multiple report formats** — Markdown, HTML, JSON.
- **Device capture/bridge backends** — srcpy, x11grab, kmsgrab, and related capture tools (cmd/helixqa-*, pkg/capture, pkg/bridge).
- **Vision analysis** — GoCV mechanical vision + LLM Vision, with GPU-accelerated comparison (LPIPS, DreamSim).
- **Autonomous QA session** — 4-phase LLM-driven exploration.

Working Directory & Build System

This is a single Go module. Build and test from the module root.

```
# Build
go build ./...
go build -o bin/helixqa ./cmd/helixqa
make build

# Test
go test ./... -count=1
make test-race
make test-cover
go test -v ./pkg/testbank/ -count=1
go test -v -run TestX ./pkg/orchestrator/

# Static analysis / format
go vet ./...

# all packages
# the CLI (or: make build)
# → bin/helixqa

# all tests (or: make
# with -race
# coverage → coverage.html
# one package
# one test

# (or: make vet)
```

```
make lint # golangci-lint
make fmt # gofmt -w .
```

Anti-bluff gates (CONST-035 family)

```
make anti-bluff # scanner + behaviour-anchor manifest +
mutation
make anti-bluff-scan # static bluff scanner over the full
tree
make anti-bluff-anchors # behaviour-anchor manifest validator
make anti-bluff-mutation # go-mutesting full project (slow)
make challenge # run every challenges/scripts/*.sh
make qa-all # challenge scripts + all anti-bluff
gates
```

make qa-all intentionally omits vet/test because some packages need own-org replace directives (declared in helix-deps.yaml, wired via go.work) that may be absent in a clean checkout. Run go test ./... separately once those siblings are present.

Running the framework

```
bin/helixqa run --banks banks/ --platform all
bin/helixqa list --banks banks/ --platform android
bin/helixqa report --input qa-results --format html
bin/helixqa autonomous --project /path/to/project --platforms
desktop \
--env .env --timeout 30m --output qa-results/
bin/helixqa version
```

Architecture & Code Organization

```
cmd/
  helixqa/          CLI entry point (run, autonomous, http,
replay,
                    list, report, signoff, version, help)
  helixqa-bridge/    device bridge (scrcpy-class capture)
  helixqa-capture-*/ platform capture backends (linux, x11grab,
kmsggrab)
  helixqa-axtree-*/  accessibility-tree extraction (darwin,
windows)
  helixqa-lpips/     LPIPS perceptual-distance comparator
  helixqa-dreamsim/   DreamSim perceptual-similarity comparator
  helixqa-omniparser/ UI element parsing
  helixqa-uitars/     UI-TARS vision navigation
  recording-analyzer/ recorded-evidence content analysis
  ocu-*/             on-screen-content / observe utilities
pkg/
  config/            configuration types + validation (.env-
driven)
  testbank/          YAML/JSON test-bank loading, platform/
```

priority filter	
detector/	crash/ANR detection (android.go, web.go,
desktop.go)	
validator/, validators/	step validation with evidence
evidence/	centralized evidence collection
ticket/	Markdown ticket generation for AI fix
pipelines	
reporter/	report generation (Markdown/HTML/JSON)
orchestrator/	main QA pipeline coordinator
autonomous/	SessionCoordinator, PlatformWorker,
PhaseManager	
navigator/, visionnav/	navigation engine + ADB/Playwright/X11
executors	
issuedetector/	LLM-powered bug detection (visual/UX/a11y/
functional)	
session/, recordingqa/	session recording, timeline, video,
recording QA	
bridge/, bridges/, capture/	device bridges and frame capture
vision/, gpu/	vision analysis + GPU-accelerated comparison
planning/	test-plan generation (incl. Android TV
Channels)	
llm/	LLM client wiring
regression/, replay/	regression + ticket-replay surfaces

Autonomous QA Session

The autonomous session (pkg/autonomous) runs in four phases:

1. **Setup** — select LLMs via LLMsVerifier, build a feature map from project docs via DocProcessor, spawn CLI agents via LLMOrchestrator, initialize VisionEngine.
2. **Doc-Driven Verification** — platform workers verify every documented feature against the running app, capturing screenshots and video.
3. **Curiosity-Driven Exploration** — workers explore undiscovered areas, testing edge cases, empty inputs, rapid interactions, undocumented paths.
4. **Report & Cleanup** — aggregate coverage, tickets, and navigation maps into Markdown/HTML/JSON reports with linked video timestamps.

External own-org modules it integrates (consumed by reference per helix-deps.yaml, never nested submodules per §11.4.28(C)): LLMsVerifier (LLM selection/scoring), LLMOrchestrator (headless CLI agent management), LLMPProvider, VisionEngine (GoCV + LLM Vision), DocProcessor (doc loading + feature-map/coverage).

Anti-Bluff Testing Rules

- Mocks, stubs, placeholders, TODO/FIXME, and "for now" code are permitted **only** in unit tests (§11.4.27). Every other test type — integration, e2e, full-automation, autonomous QA, Challenges — exercises the real system.
- Every advertised capability is anchored to an executable test in the behaviour-anchor manifest (make anti-bluff-anchors).

- Every new gate ships with a paired §1.1 mutation that makes the gate FAIL when the invariant is broken.
- A PASS for a user-visible behaviour MUST cite captured evidence (screenshot/video/logcat/report). A green line without evidence is a bluff and a release blocker.
- `t.Skip()` requires a topology-justified reason (§11.4.3); a missing required topology is a SKIP-with-reason, never a PASS-by-default.

Test-bank conventions

Banks under `banks/` are YAML (with optional JSON peers sharing the base name). Required top-level keys: `version`, `name`, `test_cases`. Per-test-case: `id` (TC-XXX), `name`, `category`, `priority`, `platforms` (subset of `[android, android_tv, web, desktop, ios, aurora_os, harmony_os]`), `steps[]` (`name / action / expected`), `tags[]`, optional `documentation_refs[]`. Banks describe structure, not prose — strings drive LLM prompts at runtime, so avoid hardcoded user-facing English.

Configuration

All runtime settings load from a `.env` file (copy `.env.example`). Key groups: master switch (`enable/platforms/timeout/coverage target`), LLMsVerifier strategy + thresholds, provider API keys, CLI-agent pool + binary paths, vision provider + SSIM threshold, recording (`ffmpeg path, quality`), and per-platform device/URL/process settings. Credentials are never committed (§11.4.10) — load them at runtime only.

Code Style & Conventions

- **Go**: constructor injection (`NewXxx(cfg)`), table-driven tests, `*_test.go` beside source, errors wrapped with `fmt.Errorf + %w`, `context.Context` cancellation respected in scans and sessions.
- **Concurrency**: bound worker/capture parallelism; reap child processes and goroutines on cleanup; leave the target quiescent (§11.4.14).
- **Static analysis**: `go vet ./...` clean before commit.

Governance pointers

Document	Authority
<u>CONSTITUTION.md</u>	Submodule-scoped anchors inherited from the Helix Constitution
<u>CLAUDE.md</u> / <u>AGENTS.md</u> / <u>QWEN.md</u>	AI-agent operating manuals — same inheritance pointer + anti-bluff posture
<u>helix-deps.yaml</u>	Own-org dependency manifest

Document	Authority
ARCHITECTURE.md / API_REFERENCE.md	Architecture + API detail
USER_GUIDE_AUTONOMOUS.md / VIDEO_COURSE_AUTONOMOUS.md	Autonomous-QA-session tutorials

Resources

- [README.md](#) — overview, install, usage, test-bank format.
- [docs/](#) — architecture, anti-bluff posture, test-coverage matrix, behaviour anchors, user manual.
- [Makefile](#) — authoritative list of build/test/anti-bluff targets.